

Software Requirements Specification (SRS)

Koshpal – UPI Transaction Tracker & Financial Manager

Prepared by: Koshpal Team

September 5, 2025

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Audience	2
2	Overall Description	2
2.1	User Roles and Characteristics	2
3	System Features (Functional Requirements)	3
3.1	Functional Requirements	3
4	Non-Functional Requirements	3
5	System Architecture & Diagrams	4
5.1	High-Level Architecture	4
6	Data Model & Database Schema	4
6.1	Entities and Attributes	4
7	Use Cases & System Flows	4
7.1	Key Use Cases	4
8	Testing, Deployment & Maintenance	5
9	Risk Analysis	5
10	Appendices	5
10.1	Sample UPI SMS Formats	5

1 Introduction

1.1 Purpose

This SRS describes **Koshpal**, a UPI transaction tracking and personal finance mobile application (Android first, then iOS) with a web-based admin dashboard. It outlines functional and non-functional requirements, system architecture, data model, use cases, testing strategy, and deployment considerations for interns and developers.

1.2 Scope

Koshpal automatically reads UPI-related SMS messages on Android devices (via Kotlin integration), parses transactions, stores them in a backend (Node.js + MongoDB), provides categorization, visual insights, budgeting tools, reminders, bill splitting, parental controls, and an admin web dashboard for monitoring.

1.3 Audience

This document is intended for:

- Developers and interns
- QA engineers
- Product managers
- Stakeholders

2 Overall Description

Koshpal consists of a mobile application built using **React Native CLI** with **Kotlin modules** for native SMS access, and a web-based admin dashboard. The backend uses **Node.js + Express** with **MongoDB** for persistent storage. Redis is used for caching and rate-limiting. Authentication is handled via **Firebase OTP**.

2.1 User Roles and Characteristics

Role	Responsibilities
User	Register/login via phone OTP, view transactions using SMS fetching, tag/categorize, set budgets, split bills.
Admin	View platform-wide analytics, manage users, monitor suspicious activity, export reports.
Parent/Guardian	(Optional) Link to child accounts, set limits, receive reports.
Support/QA	Manage issues, view logs, assist users.

3 System Features (Functional Requirements)

3.1 Functional Requirements

ID	Requirement
FR-1	User Registration & Authentication (Firebase OTP)
FR-2	Auto-read and parse UPI SMS (Android Kotlin module)
FR-3	Transaction normalization and categorization (auto + manual)
FR-4	Manual transaction addition & editing
FR-5	Budget creation and alerts (per category & overall)
FR-6	Visual insights (monthly/weekly charts, category breakdown)
FR-7	Bill reminders & scheduled notifications
FR-8	Bill splitting & settlement tracking
FR-9	Parental controls & linked accounts
FR-10	Admin dashboard: user & transaction monitoring, reports export
FR-11	Data export (CSV) & backup features

4 Non-Functional Requirements

- **Performance:** SMS parsing should complete within <2s after receipt.
- **Scalability:** Support up to 10,000 users with sharding and horizontal scaling.
- **Security:** All communication over HTTPS; encrypt sensitive data; role-based access control.
- **Reliability:** Aim for 99.9% uptime; implement backups for MongoDB.
- **Compatibility:** Android first (API level 26+) – app will support Android 8.0 (Oreo) and above, iOS planned; web dashboard supports modern browsers.

5 System Architecture & Diagrams

5.1 High-Level Architecture

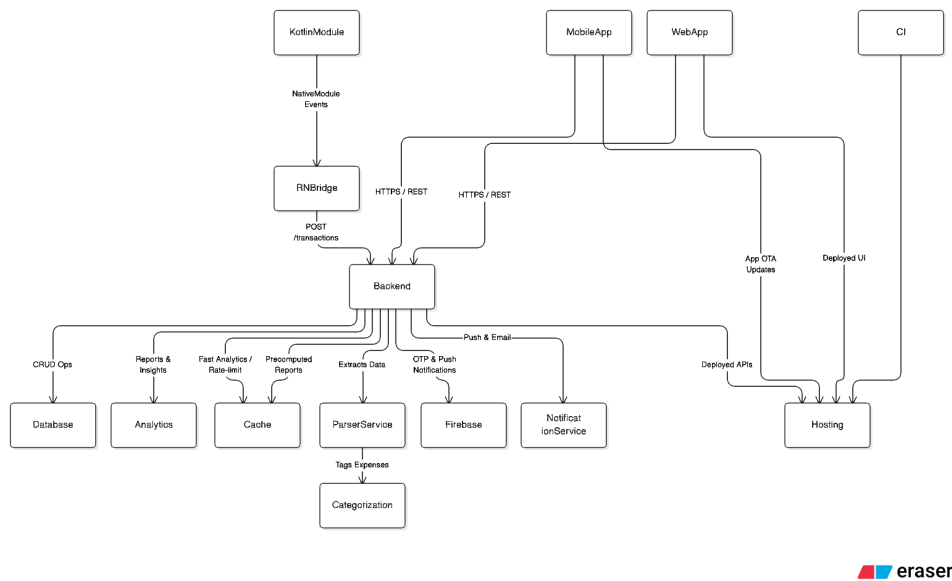


Figure 1: High-level system architecture (React Native CLI + Kotlin, Node.js backend, MongoDB, Admin Dashboard).

6 Data Model & Database Schema

6.1 Entities and Attributes

gray!20 Entity	Key Attributes
User	user_id (PK), name, phone, email, otp_verified, created_at
Transaction	txn_id (PK), user_id (FK), amount, type, merchant, category, original_text, timestamp
Budget	budget_id, user_id, category, limit_amount, period, created_at
Reminder	reminder_id, user_id, title, due_date, repeat, notified
LinkedAccount	link_id, user_id, account_idenfier, bank_name
Split	split_id, txn_id, participants, amounts, settled_flags

7 Use Cases & System Flows

7.1 Key Use Cases

- **Sign-up/Login:** User enters phone number → OTP via Firebase → verifies → account created.

- **Auto Transaction Capture:** Kotlin SMS listener captures UPI SMS → React Native → Backend → Stores transaction.
- **Budget Alerts:** System checks budgets daily and sends push notifications.
- **Bill Splitting:** User creates split request → participants confirm and settle.

8 Testing, Deployment & Maintenance

- Unit tests for parsing logic and backend services.
- Integration tests for mobile-backend interactions.
- E2E tests for signup, transaction capture, budgeting.
- Deployment via Docker + AWS ECS/Vercel.
- Monitoring via Prometheus/Grafana.

9 Risk Analysis

Risk	Impact	Mitigation
SMS Parsing Changes	High	Maintain modular regex-based parsers; fallback to manual entry.
User Denies SMS Permission	High	Provide clear onboarding and manual import option.
Data Breach	High	Encrypt sensitive fields; rotate keys; use least privilege.
Server Downtime	High	Auto-scaling, health checks, and backups.

10 Appendices

10.1 Sample UPI SMS Formats

- “Your a/c XXXXX1234 is debited by INR 499.00 on 05-09-2025 at PAYTM UPI. Avl bal: INR 1500.00”
- “TXN of INR 250.00 at ZOMATO on 05-09-2025. Ref: UPI12345. Msg: ...”